

12. Bölüm

Dizgiler

12.1 Dizgi Tanımlama

C dilinde **dizgi** (**string**), dizideki son karakteri `'\0'` yani **boş karakter** (NULL character) olan, **char** veri tipinde tek boyutlu bir dizidir. Çoğunlukla metin içerirler;

```
//Karakter Dizisi olarak dizgi tanımı:
char metin1[]={ 'C','L','a','n','g','\0' };
char metin2[]={ 'C','L','a','n','g', 0 };
// metin1 ve metin2 dizgileri 6 karakterden oluşan dizi ile başlatılmıştır.
```

Konsola metin yazmak 5.2 başlığında metinlerin çift tırnak karakteri arasında harflerin yazılmasıyla oluştuğu anlatılmıştı. Yukarıdaki metni ifade eden dizgileri uzun uzun dizi şeklinde tanımlamak yerine çift tırnak karakteri kullanılır;

```
char metin[] = "CLang"; //Kısaca Dizgi Tanımı:
```

Bu tanımda çift tırnak karakterleri kullanılarak ilk değer olarak metin kimlikli dizgiye **"Clang"** **metin değişmezi** (**string literal**) atanmıştır. Aslında karakter dizisi olan bu metin aşağıdaki iki şekilde de tanımlanabilir; Dizgiler karakter göstericisi olarak da tanımlanır. Çünkü diziler göstericileri ifade etmesi 9.6 başlığında anlatılmıştı. Bu durumda gösterici dizinin ilk elemanını gösterir;

```
char* metin3="CLang";
```

12.2 Konsola Metin Yazma

Dizgiler, karakter dizileri olduğundan döngü kullanarak ekrana aşağıdaki şekilde yazabiliriz;

```
#include <stdio.h>
int main()
{
    char metin[] = "CLang";
    //For döngüsü ile
    for (int i = 0; i < 5; i++) {
        printf("%c", metin[i]);
    }
    printf("\n");
    //while döngüsü ile boş karakteri görene kadar
    char* ptrc=&metin[0]; //char *ptr3=metin3;
    while (*ptrc!=0)
        printf("%c",*ptrc++);
    printf("\n");
    return 0;
}
```

Konsola biçimlendirilmiş veri yazmak 5.2 başlığında anlatılmıştı. Dizgileri yukarıdaki gibi uzun uzadıya döngü yazarak konsola yazma yerine döngü kullanmadan **printf** fonksiyonunun biçimlendirme metni içinde **%s** **biçim belirleyicisi** (**format specifier**) kullanarak da yazdırabiliriz.

```
#include <stdio.h>
int main() {
    char metin[]="CLang";
    printf("%s\n", metin); //Clang
    char metin1[]={ 'C','L','a','n','g','\0' };
    printf("%s\n", metin1); //Clang
    char metin2[]={ 'C','L','a','n','g', 0 };
    printf("%s\n", metin2); //Clang
    char *metin3="CLang";
    printf("%s\n", metin3); //Clang
    return 0;
}
```

12.3 Klavyeden Metin Okuma

Klavyeden metin okunacak ise ilk önce okunacak karakter sayısına bağlı olarak bir dizgi tanımlanmalıdır; `char string[20];` //20 karakterlik "dizgi" kimlikli değişken tanımı.

Dizgileri klavyeden okumak için, `printf` fonksiyonundaki gibi, `scanf` fonksiyonlarında da `%s` biçim belirleyicisi kullanılır. Dizgi için belirlenen karakter sayısından fazlası okunduğunda belleğin istenmeyen bölgelerine ulaşılacağından okunacak metnin koyulacağı dizginin boyutuna dikkat edilmelidir. Okunan değerlerin konulacağı değişkenin adresi `scanf` fonksiyonunda kullanılır. Ancak dizgiler de karakter dizileri olduğundan ve diziler de gösterici gibi kullanıldıklarından, dizgi değişkeninin önünde `&` adres işleci (address operator) kullanılmaz.

```

1  #include <stdio.h>
2  #include <string.h>
3  int main (){
4      char adi[20];
5      char soyadi[20];
6      printf("Adınızı Giriniz:");
7      scanf("%s", adi);
8      printf("Soyadınızı Giriniz:");
9      scanf("%s", soyadi);
10     printf("Girdiğiniz değerler:");
11     printf("%s-%s\n", adi,soyadi);
12     printf("Adınızı ve Soyadınızı Giriniz:");
13     scanf("%s %s", adi, soyadi);
14     printf("Girdiğiniz değerler:");
15     printf("%s-%s\n", adi,soyadi);
16     char tamAdi[40];
17     printf("Adınızı ve Soyadınızı Tam Giriniz:");
18     scanf("%s", tamAdi);
19     printf("Girdiğiniz değer:");
20     printf("%s\n", tamAdi);
21     return 0;
22 }
23 /* Program Çıktısı
24 Adınızı Giriniz:Ilhan
25 Soyadınızı Giriniz:Ozkan
26 Girdiğiniz değerler:Ilhan-Ozkan
27 Adınızı ve Soyadınızı Giriniz:Ilhan Ozkan
28 Girdiğiniz değerler:Ilhan-Ozkan
29 Adınızı ve Soyadınızı Tam Giriniz:Ilhan OZKAN
30 Girdiğiniz değer:Ilhan
31 */

```

Yukarıdaki kodu incelediğimizde 18. satırda girilen değere ilişkin çıktı 29. satırda gösterilmiştir. Burada girilen "Ilhan Ozkan" metnine karşılık `tamAdi` değişkenine konulan değer sadece "Ilhan" olmuştur. Bu da 30. satırda görülmektedir. Bunu düzeltmenin yolu `fgets()` fonksiyonunu kullanmaktır. Bu fonksiyona dizginin boyutunu parametre olarak verirsiniz. Belirtilen boyuttan fazla karakter okumaz. Ayrıca sadece klavyeden (`stdin`) değil, dosyalardan da veri okuyabilir. İleride anlatılacaktır

```

#include <stdio.h>
int main() {
    char isim[20];
    // gets(isim);
    // gets TEHLİKELİ. Artık C standartlarında yok. Kullanıcı 20'den fazla karakter girerse
    //   ↪ program çöker/hacklenebilir.
    // GÜVENLİ YÖNTEM: fgets
    printf("Adınızı giriniz: ");
    // fgets(dizi_adı, maksimum_boyut, giriş_kaynağı)
    if (fgets(isim, sizeof(isim), stdin) != NULL) {
        printf("Merhaba, %s", isim);
    }
    return 0;
}

```

12.4 Çok Kullanılan Dizgi Fonksiyonları

C dili, dizgiler üzerinde çeşitli işlemler gerçekleştirmek için standart `string.h` başlığına sahiptir. Bu kütüphanedeki fonksiyonları kullanırken göstericilerin bir karakter dizinini işaret ettiği unutulmamalıdır.

Fonksiyon	Açıklama
<code>strcat()</code>	Bir dizgiyi diğerinin sonuna ekler
<code>strchr()</code>	Dizgi içinde aranan bir karakterin ilk bulunanına ait göstericiyi döndürür.
<code>strcmp()</code>	ASCII olarak iki dizgiyi karşılaştırır.
<code>strcpy()</code>	Hafızada yer alan iki dizgiden birinin içeriğini diğerine kopyalar.
<code>strlen()</code>	Dizginin karakter uzunluğunu verir.
<code>strrchr()</code>	Dizgi içinde aranan bir karakterin son bulunanına ait göstericiyi döndürür.
<code>strstr()</code>	Dizgi içinde aranan ikinci bir dizginin ilk bulunanına ait göstericiyi döndürür.
<code>strtok()</code>	Dizgiyi verilen ayraçlara bağlı olarak alt dizgilere böler

Tablo 12.1: Çok Kullanılan Dizgi Fonksiyonları

§12.4.1 strlen() fonksiyonu

Bu fonksiyon dizgideki karakter sayısını verir.

```
#include <stdio.h>
#include <string.h>
int main(){
    char *metin = "Merhaba";
    printf("Dizgi: %s\n", metin);
    printf("Dizgi Uzunluğu: %ld", strlen(metin));
    return 0;
}
```

§12.4.2 strcpy() fonksiyonu

Bu fonksiyon, ikinci parametredeki dizgiyi, birinci parametreye kopyalar. Burada dikkat edilmesi gereken husus; hedef gösterilen dizginin karakter sayısının veya göstericinin işaret ettiği bellek bölgesinin büyüklüğünün en az kaynak kadar olması gerektiridir.

```
#include <stdio.h>
#include <string.h>
int main(){
    char* ptrToDegimmezDizgi = "Ne Haber?";
    char bosDizgi[20]; //Üzerine kopyalanacak metnin karakter sayısı 20 kabul edilmiş!
    char* ptr;
    strcpy(bosDizgi, ptrToDegimmezDizgi);
    printf("%s\n", bosDizgi);
    //strcpy(ptr, ptrToDegimmezDizgi);
    /* HATA: ptr sadece adres tutan değişkendir.
    Şu an için hiçbir dizgiyi göstermiyor! */
    ptr = bosDizgi;
    strcpy(ptr, ptrToDegimmezDizgi);
    printf("%s", ptr);
    return 0;
}
```

Bu fonksiyonu kullanmak yerine kendimiz de dizgi kopyalayan fonksiyon yazabiliriz;

```
#include <stdio.h>
void copyString(char*, char*);
int main(){
    char str1[100], str2[100]="Deneme Metni";
    printf("str1: %s \nstr2: %s\n", str1, str2);
    copyString(str1, str2);
    printf("str1: %s \nstr2: %s\n", str1, str2);
    return 0;
}
void copyString(char* hedef, char* kaynak){
    int i = 0;
    for(i = 0; kaynak[i]!='\0'; i++)
```

```

        hedef[i] = kaynak[i];
        hedef[i] = '\0';
    }

```

§12.4.3 strcat() fonksiyonu

Bu fonksiyon, parametre olarak girilen iki dizgiyi, birinci parametreye birleştirir.

```

#include <stdio.h>
#include <string.h>
int main() {
    char adi[] = "Ilhan";
    char* soyadi = "OZKAN";
    char adiSoyadi[50]; /* Birleştirme sonrası oluşacak metnin
    karakter sayısı 50 kabul edilmiş! */
    strcat(adiSoyadi, adi);
    strcat(adiSoyadi, " ");
    strcat(adiSoyadi, soyadi);
    printf("%s", adiSoyadi); // Çıktı: Ilhan OZKAN
    return 0;
}

```

§12.4.4 strchr() fonksiyonu

Bu fonksiyon, birinci parametre olarak verilen dizgide, ikinci parametrede verilen karakteri arar. Bulunamaz ise bu fonksiyon boş gösterici (NULL Pointer) döndürür.

```

#include <stdio.h>
#include <string.h>
int main() {
    char dizgi[] = "Merhaba!";
    char aranan='h';
    char* ptr = strchr(dizgi, aranan);
    if (ptr != NULL) {
        printf("'%' karakteri \"%s\" dizgisinde %ld indistedir.\n",
        *ptr, dizgi, ptr - dizgi);
    } else {
        printf("'%' karakteri \"%s\" dizgisinde bulunamadı.\n",
        aranan, dizgi);
    }
    return 0;
}

```

§12.4.5 strcmp() fonksiyonu

Bu fonksiyon, parametre olarak girilen iki dizginin, karakterlerini karşılıklı olarak karşılaştırır. Karşılaştırılan iki karakter birbirinden farklı ise ASCII kodu farklarını döndürür. Tüm karakterler aynı ise sıfır döndürür.

```

#include <stdio.h>
#include <string.h>
int main() {
    char str1[] = "ARMUT";
    char str2[] = "armut";
    int result = strcmp(str1, str2);

    if (result == 0) {
        puts("İki metin de eşit.");
    } else if (result < 0) {
        puts("Birinci metin ikincisinden küçük");
    } else {
        puts("İkinci metin birincisinden küçük");
    }
    return 0;
}
// Çıktı: Birinci metin ikincisinden küçük

```

Alternatif olarak karşılaştırma fonksiyonunu aşağıdaki gibi yazabiliriz;

```
#include <stdio.h>
int compareString(const char*, const char*);
int main() {
    char str1[] = "Elma";
    char str2[] = "Elma2";
    int result = compareString(str1, str2);
    if (result == 0)
        printf("İki Metin Aynıdır.\n");
    else if (result < 0)
        printf("Birinci Metin İkincisinden Küçüktür:%d\n", result);
    else
        printf("Birinci Metin İkincisinden Büyüktür:%d\n", result);
    return 0;
}
int compareString(const char* metin1, const char* metin2) {
    while (*metin1 == *metin2) {
        if (*metin1 == '\0') return (0);
        metin1++;
        metin2++;
    }
    return (*metin1 - *metin2);
}
// Birinci Metin İkincisinden Küçüktür:-50
```

12.5 Karakter Fonksiyonları

C dilinde, standart olan bir başka başlık dosyası olan `ctype.h` içinde karakter fonksiyonları bulunur. Bu başlıkta dizgileri oluşturan karakterler üzerinde işlem yapmak için kullanılır. Karakterler aşağıdaki şekilde gruplandırılmıştır;

- ♦ **Alfasayısal (alphanumeric)**: Hem rakam, hem de harf barındıran karakter kümesidir.
- ♦ **Beyaz boşluk**: boşluk (' '), yeni satır ('\n'), satır başı ('\r'), alt satır ('\v'), ve tab ('\t') karakterlerinin oluşturduğu kümedir.
- ♦ **Onaltılık Sayı Rakamları (hexadecimal digit)**: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, a, b, c, d, e, f

Aşağıda bu başlık dosyasında yer alan birçok fonksiyon yer almaktadır;

Fonksiyon	Açıklama
<code>int isalnum(int c)</code>	Verilen karakter kodunun alfa sayısal olup olmadığını kontrol eder.
<code>int isalpha(int c)</code>	Verilen karakter kodunun harf olup olmadığını kontrol eder.
<code>int iscntrl(int c)</code>	Verilen karakter kodunun kontrol karakteri olup olmadığını kontrol eder.
<code>int isdigit(int c)</code>	Verilen karakter kodunun rakam olup olmadığını kontrol eder.
<code>int isgraph(int c)</code>	Verilen karakter kodunun konsola yazdırılabilir karakteri olup olmadığını kontrol eder.
<code>int islower(int c)</code>	Verilen karakter kodunun küçük harf olup olmadığını kontrol eder.
<code>int isprint(int c)</code>	Verilen karakter kodunun boşluk dahil konsola yazılabilir olup olmadığını kontrol eder.
<code>int ispunct(int c)</code>	Verilen karakter kodunun noktalama işareti olan karakterler olup olmadığını kontrol eder.
<code>int isspace(int c)</code>	Verilen karakter kodunun beyaz boşluk olup olmadığını kontrol eder.
<code>int isupper(int c)</code>	Verilen karakter kodunun büyük harf olup olmadığını kontrol eder.
<code>int isxdigit(int c)</code>	Verilen karakter kodunun onaltılık sayı rakamı olup olmadığını kontrol eder.
<code>int isblank(int c)</code>	Verilen karakter kodunun boşluk karakteri olup olmadığını kontrol eder.
<code>int toupper(int c)</code>	Verilen karakterin büyük harfinin kodunu döner.
<code>int tolower(int c)</code>	Verilen karakterin küçük harfinin kodunu döner.

Tablo 12.2: Karakter Fonksiyonları

Bu fonksiyonlar kullanılarak küçük-büyük harf ayrımı yapmadan metinleri karşılaştıran bir program örneği verilmiştir;

```
#include <stdio.h>
```

```

#include <string.h>
#include <ctype.h>
int compareString(char*, char*);
int main() {
    char str1[] = "Elma";
    char str2[] = "elma";
    int result = compareString(str1, str2);
    if (result == 0)
        printf("İki Metin Aynıdır.\n");
    else if (result < 0)
        printf("Birinci Metin İkincisinden Küçüktür.\n");
    else
        printf("İkinci Metin Birincisinden Küçüktür.\n");
    return 0;
}
int compareString(char* hedef, char* kaynak){
    int i = 0;
    for (i = 0; hedef[i]; i++) //hedefi küçük harfe çevir.
        hedef[i] = tolower(hedef[i]);
    for (i = 0; kaynak[i]; i++) //kaynağı küçük harfe çevir.
        kaynak[i] = tolower(kaynak[i]);
    return strcmp(hedef, kaynak);
}

```

12.6 Metinden İlkel Veri Tiplerine Dönüşüm

Standart başlık dosyalarından olan `stdlib.h` başlık dosyasında daha önce işlediğimiz rastgele fonksiyonların yanında metinden ilkel veri tiplerine dönüşüm sağlayan bazı fonksiyonlar da bulunur.

Fonksiyon	Açıklama
<code>double atof(const char*)</code>	Dizgi olarak verilen metni double tipine dönüştürür ve geri döndürür.
<code>int atoi(const char*)</code>	Dizgi olarak verilen metni int tipine dönüştürür ve geri döndürür.
<code>long atol(const char*)</code>	Dizgi olarak verilen metni long tipine dönüştürür ve geri döndürür.
<code>double strtod(const char*, char**)</code>	Dizgi olarak verilen metni double tipine dönüştürür ve geri döndürür. İkinci parametre ise metinde kayan noktalı sayıya çevrilmeyen ilk karakterin göstericisidir.
<code>long strtol(const char*, char**, int)</code>	Dizgi olarak verilen metni long tipine dönüştürür ve geri döndürür. İkinci parametre ise metinde kayan noktalı sayıya çevrilmeyen ilk karakterin göstericisidir. Üçüncü parametre ise metnin hangi sayı tabanında sayı içerdiğidir. Bu 2, 8, 10, 16 olabilir.

Tablo 12.3: Metinden İlkel Veri Tiplerine Dönüşüm Fonksiyonları

Aşağıda bu fonksiyonları içeren bir program örneği verilmiştir.

```

#include <stdio.h>
#include <stdlib.h>
int main() {
    char tamsayiMetin[]="-23234";
    char kayannoktalisayiMetin[]="+2.3234e-1";
    char metin[]="-12,3e-2'nin karesi";
    int i = atoi(tamsayiMetin);
    printf("Tamsayı: %d\n",i); // Tamsayı: -23234
    double d= atof(kayannoktalisayiMetin);
    printf("Kayan Noktalı Sayı: %lf\n",d);
    // Kayan Noktalı Sayı: 0.232340
    double dm;
    char* gerikalanmetin;
    dm= strtod(metin,&gerikalanmetin);
    printf("Metinden Alınan Sayı: %lf\n",dm);
}

```

```
// Metinden Alınan Sayı: -12.000000
printf("Geri Kalan Metin: %s\n",gerikalanmetin);
// Geri Kalan Metin: ,3e-2'nin karesi
return 0;
}
```

12.7 Arama Tabloları

Arama tabloları LUT (**lookup tables**), önceden hesaplanmış belirli değerlerle doldurulmuş dizilerdir. Uzun iç içe **if**, **if-else** veya **switch** ifadeleri yerine, bir programının verimliliğini artırmak için bu tablolar kullanılabilir.

```
int kareler[10] = {1, 4, 9, 16, 25, 36, 49, 64, 81, 100};
float karekokler[5] = {1.0, 1.41421, 1.7320, 2.0, 2.2360};
char* renkler[] = {"Beyaz", "Mavi", "Yeşil", "Kırmızı", "Siyah"};
```

Örnek olarak verilen bu tablolarda her seferinde hesaplama yapmak yerine önceden hesaplanmış değerleri tutarız. Arama tabloları genellikle dizgilerden oluşur. Aşağıda örnek bir program verilmiştir;

```
#include <stdio.h>
enum renlerenum {BEYAZ,MAVI,YESIL,KIRMIZI,SIYAH};
int main() {
    int kareler[10] = {1, 4, 9, 16, 25, 36, 49, 64, 81, 100};
    for (int i = 0; i < 10; i++){
        printf("%d \tKaresi(%d): %d\n", i+1, i+1, kareler[i]);
    }
    float karekokler[5] = {1.0, 1.41421, 1.7320, 2.0, 2.2360};
    for (int i = 0; i < 5; i++){
        printf("%d \tKarekökü(%d): %0.4f\n", i+1, i+1, karekokler[i]);
    }
    char* renkler[] = {"Beyaz", "Mavi", "Yeşil", "Kırmızı", "Siyah"};
    int secim=YESIL;
    printf("Seçtiğiniz renk: %s",renkler[YESIL]);
    return 0;
}
/*Program Çıktısı:
1      Karesi(1): 1
2      Karesi(2): 4
3      Karesi(3): 9
4      Karesi(4): 16
5      Karesi(5): 25
6      Karesi(6): 36
7      Karesi(7): 49
8      Karesi(8): 64
9      Karesi(9): 81
10     Karesi(10): 100
1      Karekökü(1): 1.0000
2      Karekökü(2): 1.4142
3      Karekökü(3): 1.7320
4      Karekökü(4): 2.0000
5      Karekökü(5): 2.2360
Seçtiğiniz renk: Yeşil
*/
```

12.8 Düşün ve Çöz

- ◊ **Düşün:** C dilinde dizgilerin sonunda bulunan boş karakter (**NULL Character**) karakterinin fonksiyonlar (**strlen**, **strcpy** vb.) için önemi nedir?
- ◊ **Düşün:** **gets()** fonksiyonunun C standartlarından (C11) neden kaldırıldığını ve güvenli alternatifinin ne olduğunu araştırınız.
- ◊ **Düşün:** **char s[] = "kitap";** ile **char *s = "kitap";** tanımlarının bellekte (stack vs read-only data) tutulma farkını açıklayınız.
- ◊ **Çöz:** **strcat** fonksiyonunu kullanmadan, iki ayrı dizgiyi üçüncü bir dizgiye birleştiren kendi fonksiyonunuzu yazınız.

- ◊ **Çöz:** Kullanıcının girdiği bir cümlenin içindeki kelime sayısını (boşlukları sayarak) bulan bir fonksiyon yazınız.
- ◊ **Çöz:** Standart kütüphanedeki `strcmp` fonksiyonunu kullanmadan, iki dizgiyi karşılaştırıp eşit olup olmadıklarını dönen kendi fonksiyonunuzu yazınız.